Projet de développement
LU2IN013

Groupe-ROBOCAR

Rim EL BROUSH

Chris RHAYEM

Isabela CALDEIRA RIBREIRO CRUZ

Cyril MERLIN

Yanis MAAFA

Eliott MOURRE

Introduction du sujet

Les progrès récents en intelligence artificielle et en robotique ont permis de développer des robots capables d'interagir de manière autonome avec leur environnement. Même lorsque les comportements restent simples, comme avancer, éviter un obstacle ou suivre une cible, ces mécanismes constituent la base de systèmes plus complexes utilisés aujourd'hui dans de nombreux domaines. On retrouve ce type de robot dans la logistique, la surveillance, l'assistance, ou encore dans certains systèmes de véhicules autonomes. Notre projet s'inscrit dans ce contexte : il consiste à concevoir et tester un robot mobile capable de se déplacer dans un environnement, de détecter des obstacles et d'adopter différentes stratégies de navigation. Un tel robot pourrait, à plus grande échelle, être utilisé pour des tâches de transport, de patrouille ou d'exploration dans un espace fermé.

Mode d'emploi

Pour utiliser le projet, il faut :
1.récupérer les fichiers du dépôt,
2.se placer dans le dossier principal roboCar,

Le programme peut ensuite être exécuté en simulation 2D avec le fichier main.py, ou en simulation 3D avec le fichier main3D.py. La version 2D permet d'observer facilement les déplacements du robot et ses interactions avec les obstacles. La version 3D propose une représentation plus visuelle du même comportement dans un environnement enrichi.
Le robot commence par exécuter une trajectoire en forme de carré, puis il passe à une phase d'évitement d'obstacles. Enfin, une dernière phase met en scène un jeu du chat et de la souris, dans lequel le robot poursuit une cible mobile. Une version main_reel.py existe également pour le robot réel, mais elle nécessite le matériel correspondant et les bibliothèques associées.
L'utilisateur n'a pas besoin de programmer pour lancer ces démonstrations il suffit d'exécuter le fichier voulu.

Exemples d'exécution dans le terminal :
cd roboCar
python3 main.py
python3 main3D.py

Schéma fonctionnel

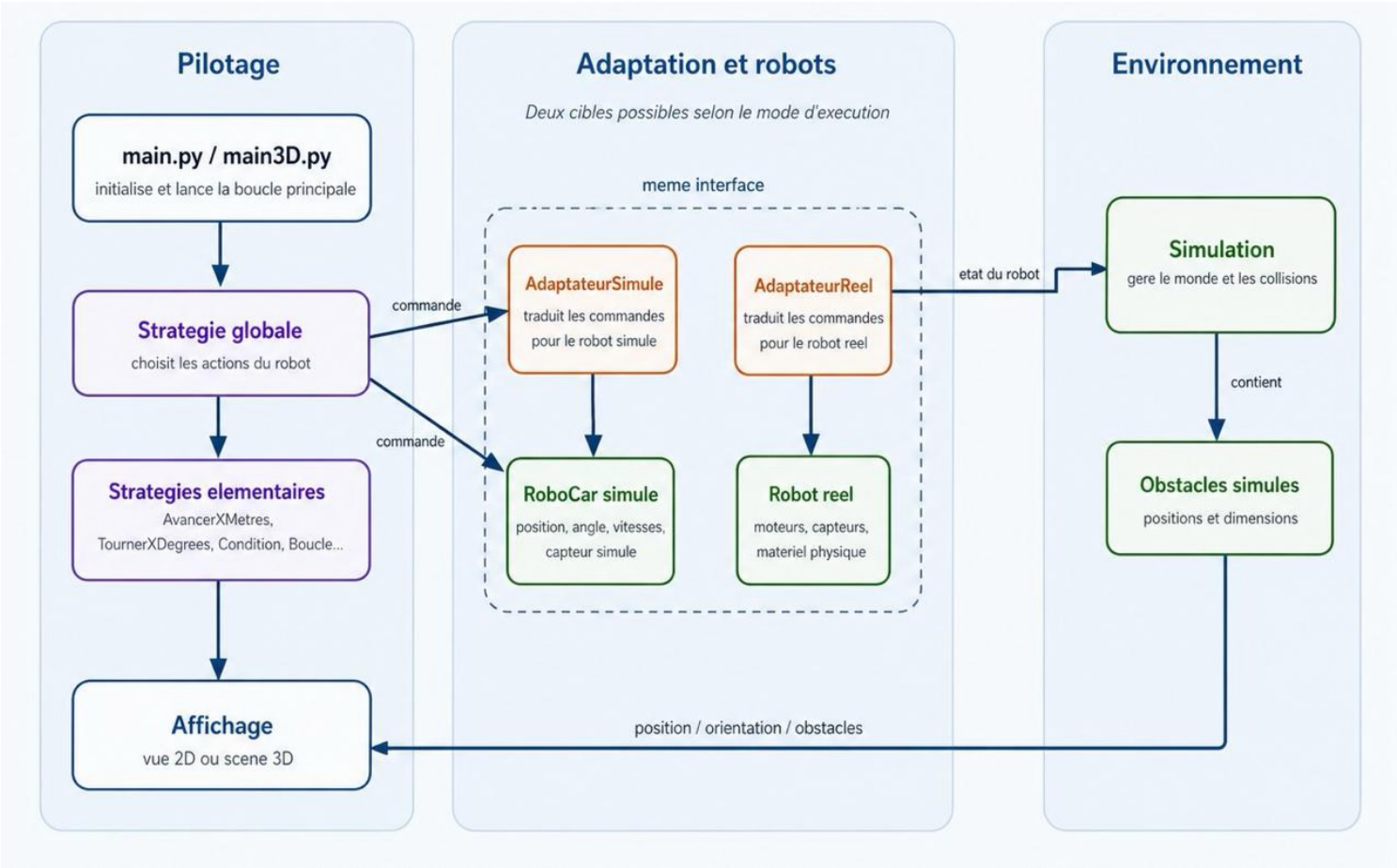

Figure 1. Schéma fonctionnel du projet.

Ce schéma montre l'organisation générale du fonctionnement du projet. Le point de départ est le programme principal, qui crée les différents modules nécessaires au lancement de l'application. Ensuite, la stratégie décide du comportement du robot, par exemple avancer ou tourner, en fonction de l'objectif fixé. Les commandes passent par un adaptateur, qui sert d'intermédiaire entre la logique de contrôle et le robot utilisé. Dans le cas de la simulation, le robot évolue dans un environnement contenant des obstacles, et la simulation vérifie les collisions. Enfin, l'affichage permet de visualiser à chaque instant la position du robot et l'état de l'environnement.

Schéma d'architecture

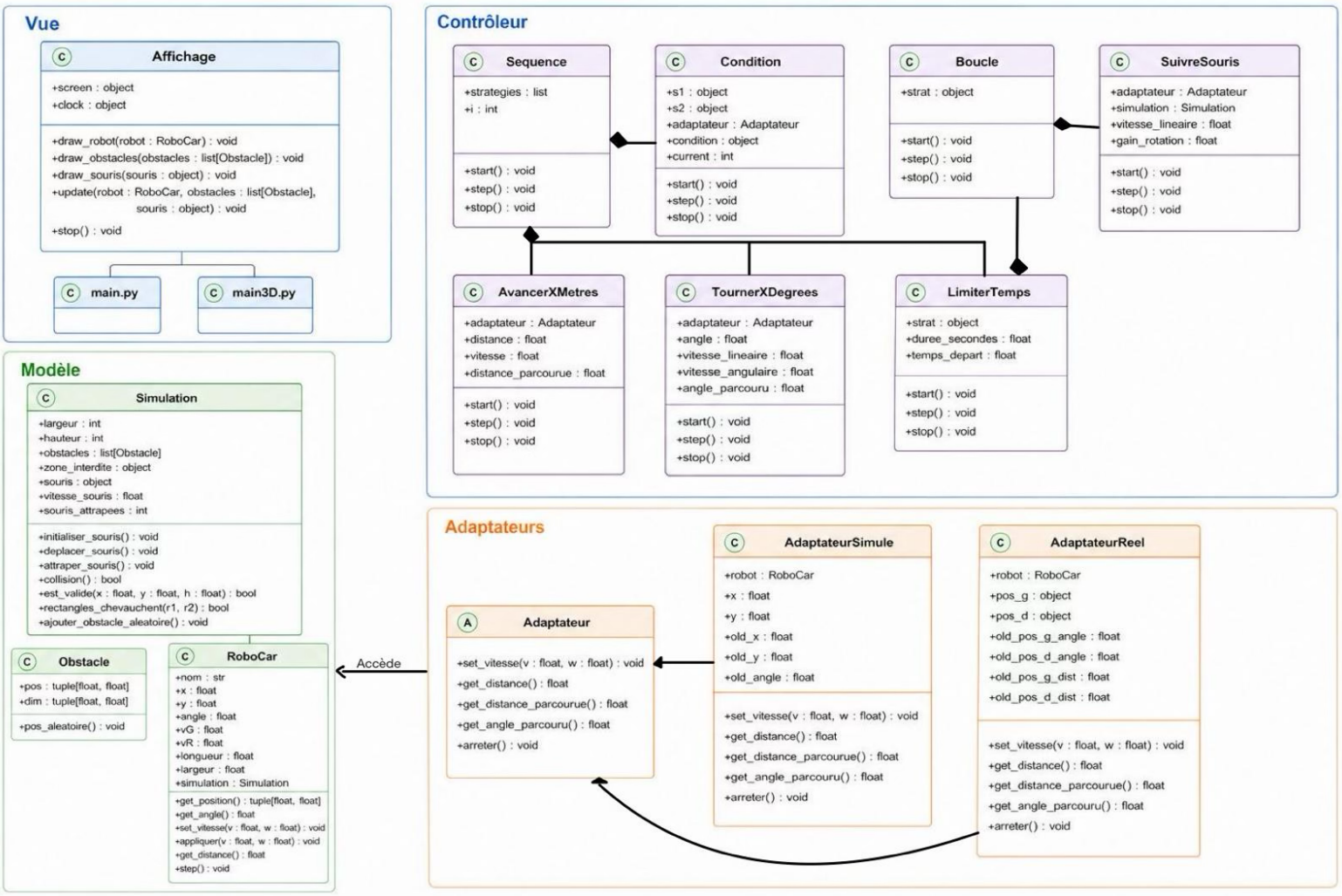

Figure 2. Diagramme UML représentant l'organisation des classes d'un système de contrôle du robot.

Ce diagramme montre l'organisation des différentes classes d'un système permettant de contrôler le robot, soit en simulation soit en conditions réelles. Il distingue plusieurs parties : la gestion des données du robot et de son environnement, l'affichage graphique, et les éléments qui pilotent les actions du robot comme avancer, tourner ou suivre une cible. Les adaptateurs permettent de relier ces commandes soit à une simulation, soit à un robot réel, sans changer le reste du système.

Équations régissant les mouvements du robot en simulation

Le robot simulé est modélisé comme un robot à deux roues motrices. Chaque roue possède une vitesse : vG pour la roue gauche et vR pour la roue droite. À partir de ces deux vitesses, on calcule la vitesse linéaire et la vitesse angulaire du robot.

$$v = \frac{vR + vG}{2}$$

$$\omega = \frac{vR - vG}{L}$$

où L représente l'écartement entre les deux roues (WHEEL_BASE).

À chaque étape de simulation, la position et l'orientation du robot sont mises à jour par les équations suivantes :

$$x_{suivant} = x + v\cos(\theta) \times pas$$

$$y_{suivant} = y + v\sin(\theta) \times pas$$

$$\theta_{suivant} = \theta + \omega \times pas$$

où :
x et y sont les coordonnées du robot,
θ est son orientation,
pas représente l'intervalle de mise à jour utilisé dans la simulation.

Pistes d'amélioration du logiciel

• Ajouter une stratégie d'évitement d'obstacles plus avancée, afin d'éviter les blocages dans certaines situations complexes.
• Améliorer la précision de la simulation en prenant davantage en compte la rotation réelle du robot dans la détection de collisions.
• Enrichir l'environnement avec plusieurs types d'objets ou de capteurs pour rendre les scénarios plus réalistes.
• Ajouter une interface permettant à l'utilisateur de choisir facilement la stratégie à exécuter.